



Πανεπιστήμιο Πειραιώς, Τμήμα Πληροφορικής
Προπτυχιακό Πρόγραμμα Σπουδών
Ακαδ. έτος 2025-26 (εαρινό εξάμηνο)



ΑΝΑΛΥΤΙΚΗ ΔΕΔΟΜΕΝΩΝ

Απαλλακτική Εργασία (ομάδες των 1-2 ατόμων)

<p23033> - <Ερμής Γρίβας> - <ermis.grivas@gmail.com>

TABLE OF CONTENTS

1. DATA PREPROCESSING.....	2
2. DATA CLUSTERING.....	4
3. CLASSIFICATION (PREDICT HOUSESTYLE).....	7
4. REGRESSION (PREDICT SALEPRICE).....	10

1. DATA PREPROCESSING

Data preprocessing started by observing the initial dataset to find any missing values. The missing values plot showed the following omissions in the provided dataset:

```
"""
The following missing data was noticed:

MiscFeature (54/1460)
Fence (281)
PoolQC (7)
Garage Related (1379)
FireplaceQu (770)
Electrical (1459)
Basement Related (1422-1423)
MasVnrArea (1452)
MasVnrType (588)
Alley (91)
LotFrontage (1201)
"""
```

A number of different methods were then utilised to fill missing values, including:

- Class-based imputation
- Median imputation
- Mode imputation

Additionally, we observed that some columns with missing data correspond to elements that don't exist in that particular house (for example, a house with no pool having NA in the PoolQC column). Those columns were filled with "None".

```
# Class-based Imputation for columns with a low number of missing values

data = train.copy()
data["Class"] = pd.cut(
    data["MSSubClass"],
    bins=[0, 20, 30, 40, 45, 50, 60, 70, 75, 80, 85, 90, 120, 160, 180, 190],
    labels=[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
)

data = data.drop(labels="MSSubClass", axis=1) # Since classes have been binned

# Median imputation for numeric columns
cols = ["MasVnrArea", "LotFrontage"]
for col in cols:
    group_medians = data.groupby(by="Class", observed=True)[col].transform("median")
    data[col] = data[col].fillna(group_medians)

# Mode imputation for string columns
cols = ["GarageCond", "GarageQual", "GarageFinish", "GarageYrBlt", "GarageType",
        "BsmtFinType1", "BsmtFinType2", "BsmtCond", "BsmtQual", "BsmtExposure", "Electrical"]
for col in cols:
    group_modes = (
        data.groupby(by="Class", observed=True)[col]
        .transform(lambda x: x.mode().iloc[0] if not x.mode().empty else None)
    )
    data[col] = data[col].fillna(group_modes)
```

```
"""
For the six remaining columns of missing data, these observations were made:
(1) If MasVnrArea = 0 then MasVnrArea = None
(2) If Fireplaces = 0 then FireplaceQu = NA
(3) If PoolArea = 0 then PoolQC = NA
(4) If MiscVal = 0 then MiscFeature = NA etc etc.
Missing values are related to things that don't exist (like houses not having a fireplace or
an alley etc), so they should be left with "None"
"""

# Adding the rest of the missing values (filled with "None")
cols = ["FireplaceQu", "Alley", "PoolQC", "MiscFeature", "Fence", "MasVnrType"]
for col in cols:
    data[col] = data[col].fillna("None")
```

After all the missing values were added, we proceeded with outlier detection, taking the following steps:

- IQR outlier detection for numeric columns
- Manual inspection on "strong outlier rows" (rows with 5+ outliers)
- Box-plot demonstration

Box plots:

- Ordinal encoding for quality-based features (ex. KitchenQual, ExterQual)

For ordinal encoding, we used tailor-made quality maps to assign a number to each corresponding categorical feature.

Example:

```
# Ordinal encoding for quality/condition based features
ordinal_quality_map = {
    "None": 0,
    "Po": 1,
    "Fa": 2,
    "TA": 3,
    "Gd": 4,
    "Ex": 5
}
```

IMPORTANT:

A copy of the data was made right before encoding “HouseStyle”, as its categorical version would later be needed for (3a).

```
classification_data = data.copy() # Before HouseStyle gets one-hot encoded

# Future target variable so will be one-hot encoded separately
one_hot = pd.get_dummies(data["HouseStyle"], dtype=int, drop_first=True, prefix="HouseStyle")
data = data.drop("HouseStyle", axis=1)
data = data.join(one_hot)
```

2. DATA CLUSTERING

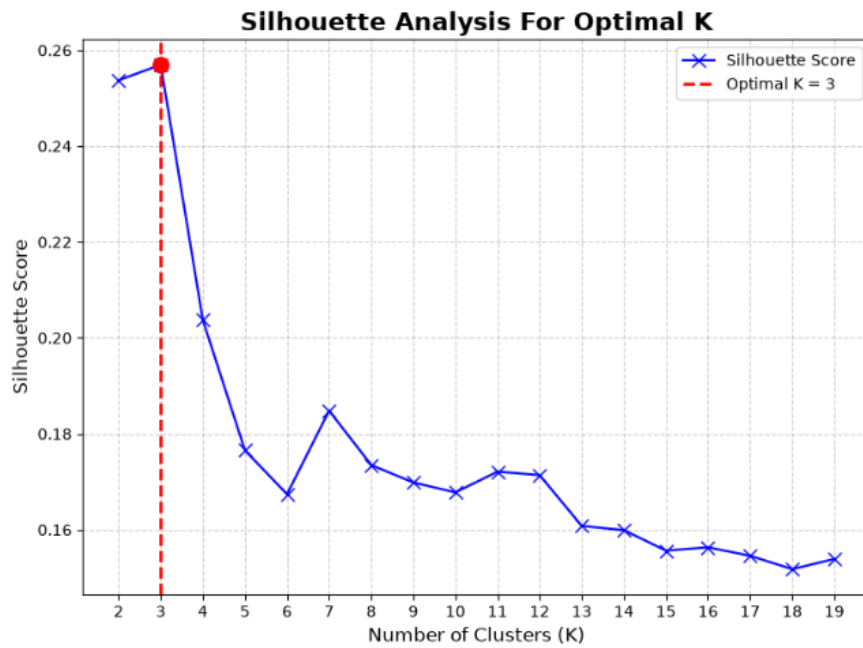
The most important step towards a successful data clustering is picking the right features. Using the preprocessed final version of the dataset, we settled on the following columns, containing natural & structural housing elements, as well as other features of high importance:

```
# Choosing natural & structural features for clustering & classification
cols = ["OverallQual", "YearBuilt", "TotalBsmtSF", "GrLivArea", "TotRmsAbvGrd", "GarageArea",
        "GarageCars", "KitchenQual", "GarageQual", "LotArea",
        "Fireplaces", "FullBath", "PoolArea", "BsmtExposure"]
```

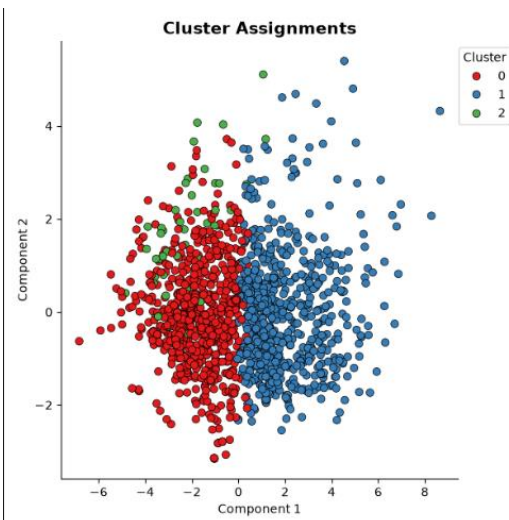
After scaling the data, we moved towards implementing 2 separate clustering algorithms:

- K-Means

Initially, we used the silhouette score method to find the optimal k for K-Means. Results showed k=3, indicating that the dataset would be separated into 3 clusters. K=2 also displayed a solid silhouette score close to k=3, but any k>3 had a sharp drop-off.



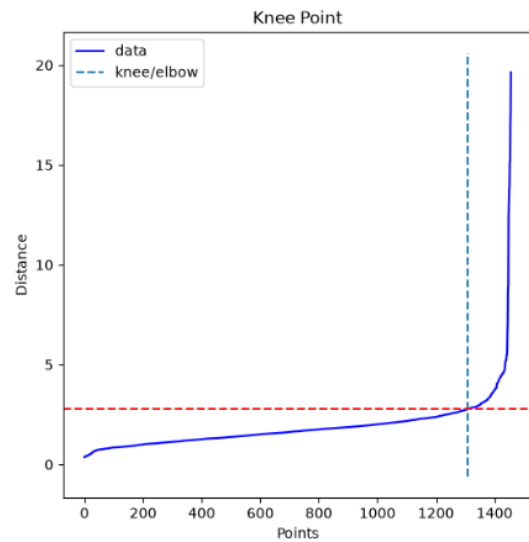
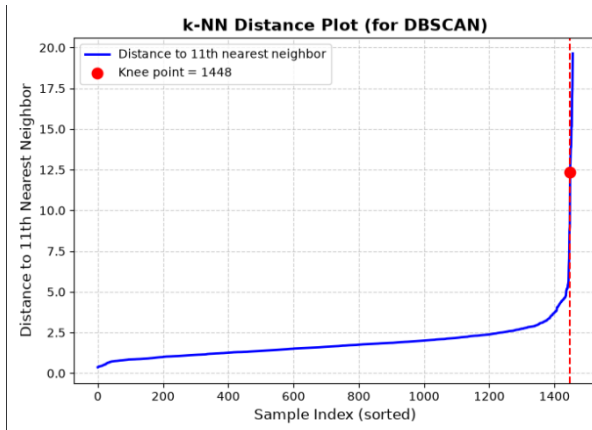
So, K-Means was implemented and then visualised through a PCA plot.



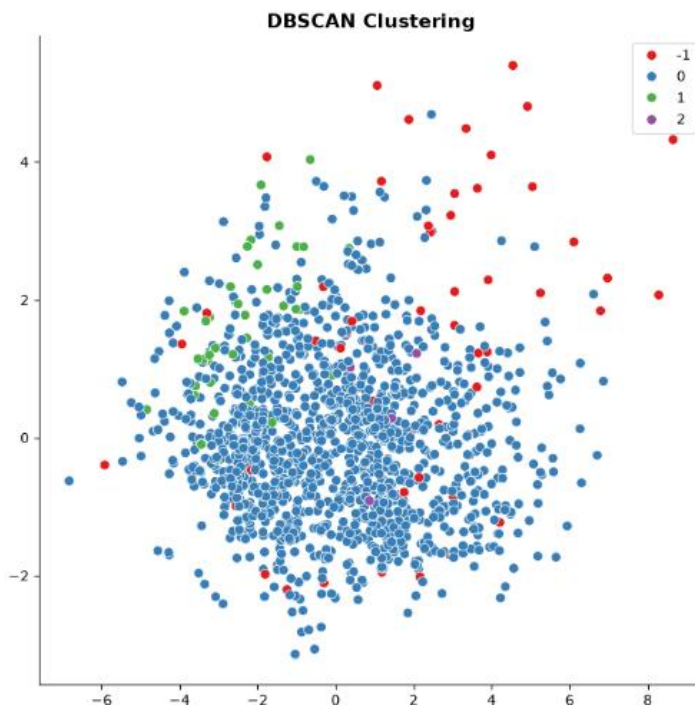
Upon observation of the plot, we can see 2 dominant clusters accounting for most of the dataset and a third cluster (green) indicating a more “niche” subgroup. The data is separated based on the quality & size of each house, with higher values obviously correlating to a more expensive SalePrice.

- DBScan

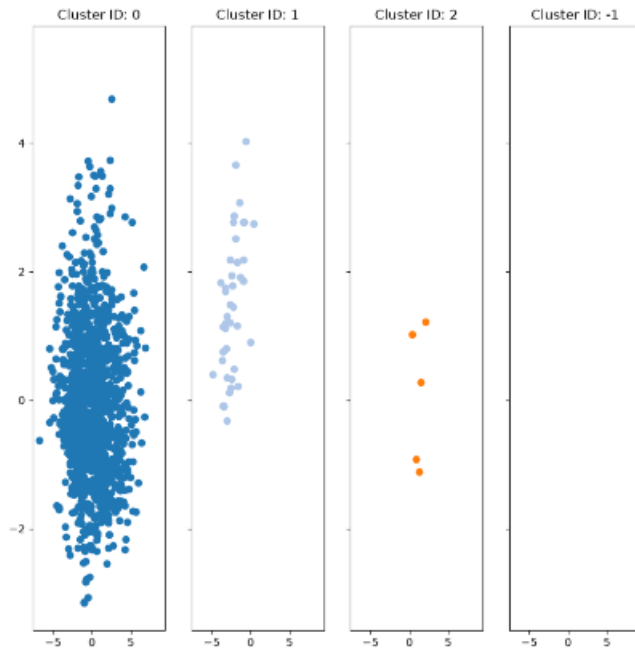
In order to run DBScan, we first had to fit the k-nearest neighbors model to calculate distances, then find the knee-point to locate eps. Both steps are visualised in the corresponding plots:



DBScan was run afterwards and produced 3 clusters, just like K-Means.

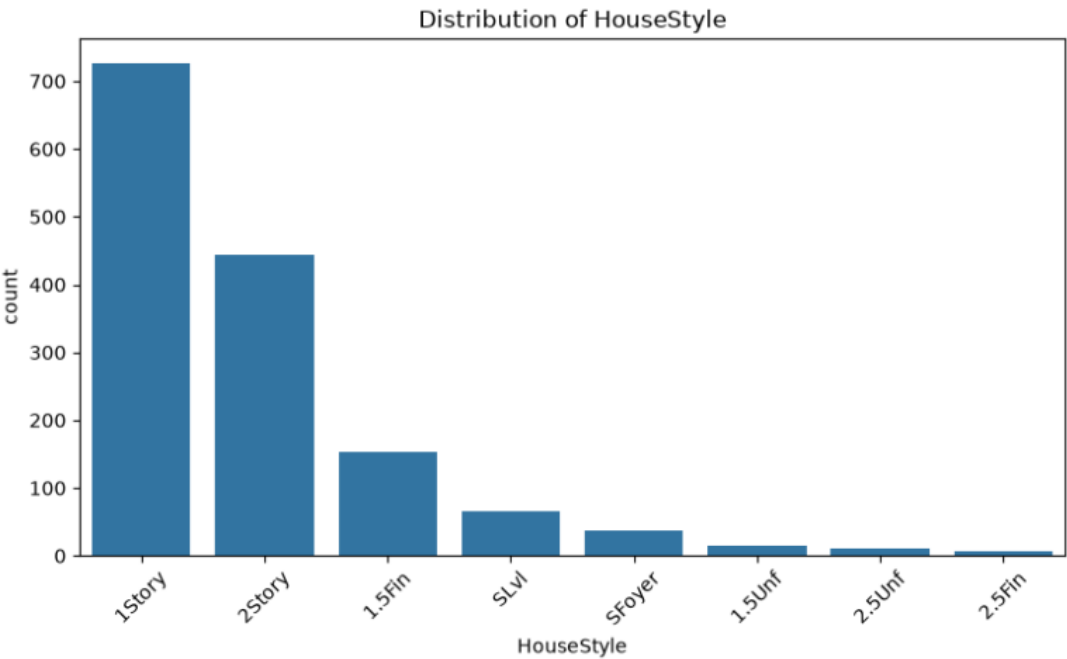


However, we observe that this time only one cluster was dominant, while the others indicate small subgroups. There's also an extra cluster ("-1") demonstrating noise/outliers.



3. CLASSIFICATION (PREDICT HOUSESTYLE)

Before moving forward with the classifier algorithms, we ran a countplot to visualise our target variable’s distribution (in this case “HouseStyle”).



From this chart it's clear that most houses belong in either the "1Story" or "2Story" class (close to 80% of the total houses), while every other class is a niche subgroup.

We used 2 different classifiers with Python's Scikit-Learn library, which produced 2 entirely different outcomes:

- Naive Bayes Classifier

Using `sklearn.naive_bayes.GaussianNB`, we trained a Naive Bayes classifier to predict a house's HouseStyle based on the same features used during clustering. Before training the classifier, we implemented a train-test split on the data. The train set (accounting for 80% of the dataset) was used for model training and the test set (the other 20%) was used for model evaluation.

However, the model proved weak with low accuracy and F1-scores across the board.

NB Accuracy: 0.5

	precision	recall	f1-score	support
1.5Fin	0.22	0.61	0.32	31
1.5Unf	0.13	1.00	0.23	3
1Story	0.87	0.45	0.59	146
2.5Fin	0.33	1.00	0.50	1
2.5Unf	0.00	0.00	0.00	2
2Story	0.81	0.56	0.66	89
SFoyer	0.22	1.00	0.36	7
SLvl	0.50	0.08	0.13	13
accuracy		0.50		292
macro avg	0.38	0.59	0.35	292
weighted avg	0.73	0.50	0.55	292

- SVM Classifier

The main difference before training the SVM classifier is that, for SVM, the data had to be scaled beforehand. SVM ended up being a much stronger predictor as the following classification report indicates:

SVM Accuracy: 0.773972602739726

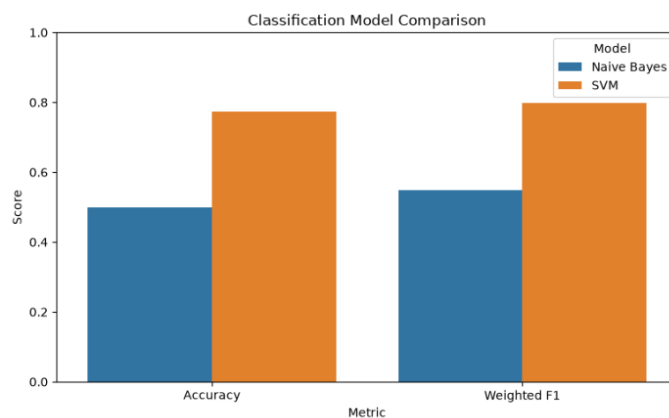
	precision	recall	f1-score	support
1.5Fin	0.59	0.77	0.67	31
1.5Unf	0.11	0.33	0.17	3
1Story	0.96	0.77	0.86	146
2.5Fin	0.33	1.00	0.50	1
2.5Unf	0.20	0.50	0.29	2
2Story	0.91	0.81	0.86	89

SFoyer	0.36	0.71	0.48	7
SLvl	0.39	0.69	0.50	13

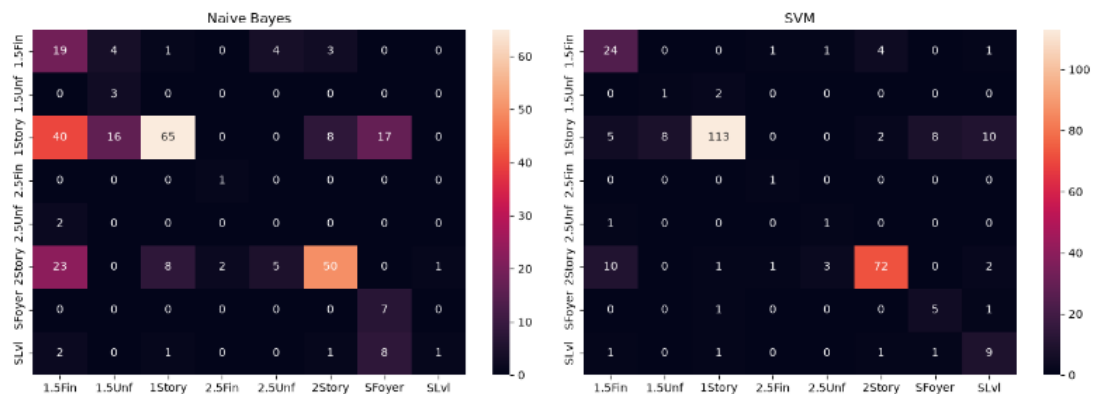
accuracy		0.77	292
macro avg	0.48	0.70	0.54
weighted avg	0.85	0.77	0.80

FURTHER COMPARISONS:

- Visualisation of the accuracy and weighted F1 difference via barchart



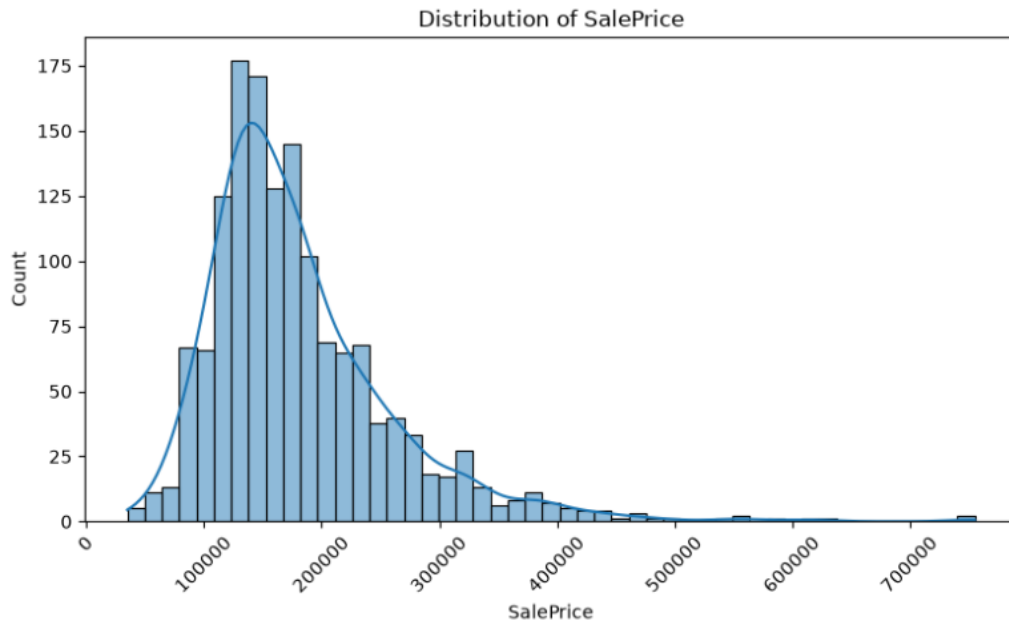
- Confusion matrices



From the confusion matrices, it's clear that the difference in accuracy comes from the SVM Classifier's ability to more correctly predict the 2 dominant classes.

4. REGRESSION (PREDICT SALEPRICE)

Before moving forward with the neural networks, we ran a histplot to visualise our target variable's distribution (in this case "SalePrice").



We then trained a FeedForward neural network to predict SalePrice based on each house's other features, using Python's

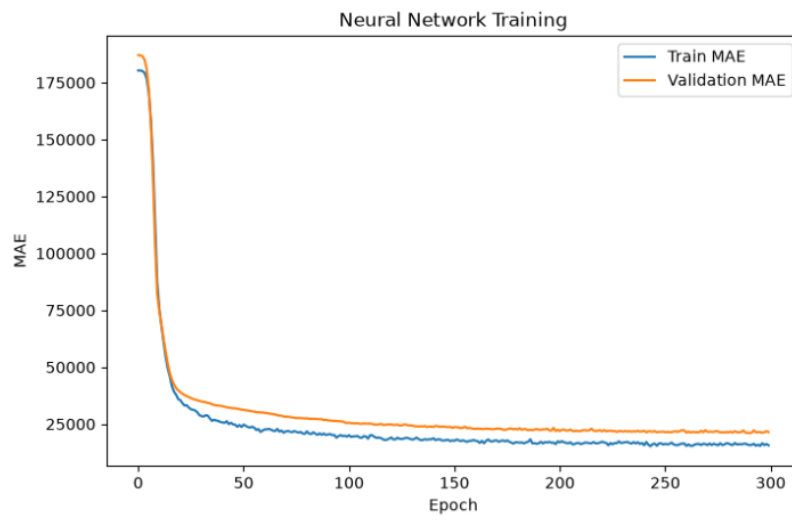
tensorflow.keras library. The process went as follows:

- Train-test split
- Scale data
- Build model
- Compile model
- Early stopping
- Train model
- Evaluate

The following results reflect the success of the model:

- MAE: 22388.955078125 (\$22.3k mean prediction error per house)
- MAPE: 0.14342717826366425 (14% mean percentage error)
- RMSE: 34987.75099945694 (\$35k mean squared error)
- R^2 : 0.8043358325958252 (prediction accuracy percentage 80%)

While the following plot demonstrates the learning process the model undergoes while training (notice the error rates keep going down):



A Transfer Learning model was not included in the project.